

LLM-Only PDDL Domain Repair with Open-Weight Models

Nader Karimi Bavandpour, Pascal Bercher

School of Computing, The Australian National University, Canberra, Australia
 {nader.karimiBavandpour, pascal.bercher}@anu.edu.au

Abstract

AI planning is concerned with finding a sequence of actions that achieves a specified goal. It relies on explicit models of the world, commonly represented in the Planning Domain Definition Language (PDDL). An active line of research investigates how errors in such models can be detected and repaired. For example, users may provide positive test plans that are solutions, and negative test plans that fail during execution. Automated repair methods then modify the PDDL model to satisfy these constraints. In this paper, we evaluate the ability of recent open-weight large language models to perform this repair task using an LLM-only approach. Our experiments show that the symbolic baseline achieves an F_1 score of .49, while the best-performing LLM reaches .87 with high reasoning effort, an absolute improvement of .38. However, that setting has a mean test pass rate of only .82, falling to .06 on the Thoughtful domain; even the best setting that includes the test traces reaches only .92. Thus, current open-weight models cannot guarantee satisfaction of the test constraints required for reliable automated model repair.

Introduction

Explainability is a central requirement for AI systems that interact with or support humans in decision making. In AI planning, this requirement is naturally addressed by the explicit representation of actions, states, and goals: planners generate solutions by reasoning over structured models of the world. Compared to black-box machine learning techniques, this explicit reasoning process makes planning inherently transparent and interpretable. However, one of the main challenges to deploying planning in practice lies in constructing the planning models themselves (Tantakoun, Muise, and Zhu 2025).

The recent success of large language models (LLMs) has motivated extensive research into their application to AI planning tasks (Guan et al. 2023; Oswald et al. 2024; Huang, Lipovetzky, and Cohn 2025; Katz et al. 2025; Huang, Cohn, and Lipovetzky 2025). A recent survey (Tantakoun, Muise, and Zhu 2025) highlights the potential of LLMs to support the construction and refinement of planning models. While verifiable planning modules remain the backbone of reliability, robustness, and explainability, LLMs can act as assistants to reduce the manual burden of defining domain models. We believe that one promising avenue is *domain repair*,

where the goal is to identify a set of modifications to a domain such that the positive traces become executable and the negative traces become non-executable.

Repairs can themselves be understood as explanations. Following Miller’s (2019) account of contrastive explanations in the social sciences, a repair answers the question of why a given trace fails in the current domain and provides a counterfactual justification of how the domain could have behaved differently. Each repair is thus not only a technical fix, but also a form of interpretable feedback to the human domain modeler.

Several symbolic approaches enforce executability constraints by automatically modifying PDDL models to satisfy them (Aineto, Jiménez, and Onaindia 2018; Gragera et al. 2025; Beriachvili and Bit-Monnot 2026; Bercher, Sreedharan, and Vallati 2025), while other work investigates the computational complexity of such repairs (Lin and Bercher 2021). We build on the setting introduced by Lin, Grastien, and Bercher (2023) and Lin et al. (2025), in which planning domains are repaired using positive, or whitelist, traces that must be valid plans and negative, or blacklist, traces that must be rendered inapplicable. They provide a symbolic algorithm that can find cardinality-minimal solutions that satisfy the blacklist and whitelist constraints. The algorithm was evaluated using precision and recall by comparing the predicted repairs with the ground-truth corrections for PDDL models that had been corrupted through the random addition or removal of action preconditions and effects.

In this paper, we investigate the ability of recent open-weight large language models of varying sizes to solve the PDDL domain-repair problem and compare their performance with the symbolic optimizer described above. We evaluate all models on the benchmark suite introduced by Lin et al. (2025), enabling a direct comparison with the symbolic approach. Our experimental setting follows the LLM-only repair formulation introduced under the same name in our earlier work (Karimi Bavandpour and Bercher 2025), but extends that study by evaluating a broader range of more recent open-weight models, allowing us to assess the current state of the art in LLM-based PDDL model repair. In addition to the domain-only NOTRACE prompt used previously, we investigate whether the WITHTRACE prompt, which also provides the LLM with the whitelist and blacklist traces, yields further performance improvements.

Planning Formalism

Since our focus is on repairing lifted PDDL domains, we introduce the lifted planning formalism. A lifted planning problem is defined as a tuple $\Pi = (\mathcal{P}, \mathcal{A}, \alpha, \mathcal{O}, s^I, s^g)$, where the domain is $\mathcal{D} = (\mathcal{P}, \mathcal{A}, \alpha)$ and the task is $\mathcal{T} = (\mathcal{O}, s^I, s^g)$.

Objects, Types, and Variables. Let $\mathcal{O}$ be the set of objects in the planning task. We consider a set of variables $\mathcal{V}$, each acting as a placeholder for an object. The type of a variable $v \in \mathcal{V}$ is written as $v|t$, where $t \in \Theta$. Every type $t \in \Theta$ is associated with the set of objects $\mathcal{O}[t] \subseteq \mathcal{O}$. We say that $t \in \Theta$ is a *subtype* of $t' \in \Theta$ iff $\mathcal{O}[t] \subseteq \mathcal{O}[t']$.

Predicates. A predicate $\mathbf{p} = P(v_1|t_1, \dots, v_k|t_k)$ is defined by a unique name P and a tuple of $k \in \mathbb{N}_0$ typed variables, written as $\text{par}(\mathbf{p}) := (v_1|t_1, \dots, v_k|t_k)$. The predicate has zero variables if $k = 0$. We denote by $\mathcal{P}$ the set of all predicates in Π .

Variable Substitution. A variable substitution function $\varrho : \mathcal{V} \rightarrow \mathcal{O}$ maps each typed variable $v|t$ to an object $\varrho(v|t) \in \mathcal{O}[t]$ of the same type t .

Facts. Given a predicate $\mathbf{p} \in \mathcal{P}$ and a substitution function ϱ , a *fact* is obtained by grounding $\mathbf{p}$, that is, by replacing each parameter $(v_1, \dots, v_k)$ with the corresponding objects given by ϱ : $f = \varrho(\mathbf{p}) = P(\varrho(v_1), \dots, \varrho(v_k))$. The set of all grounded predicates is denoted by $\mathcal{F}$, and any set of facts constitutes a *state*.

Action Schemas. Let $\mathcal{A}$ denote the set of action schemas. An action schema $\mathbf{a} = A(v_1|t_1, \dots, v_k|t_k)$ is defined by a unique name A and a tuple of k variables, written as $\text{par}(\mathbf{a}) := (v_1|t_1, \dots, v_k|t_k)$. Each schema is associated with a mapping

$$\alpha(\mathbf{a}) = (\text{prec}^+(\mathbf{a}), \text{prec}^-(\mathbf{a}), \text{eff}^+(\mathbf{a}), \text{eff}^-(\mathbf{a}))$$

representing a tuple of four sets of compatible predicates, as defined below.

Definition 1 (Compatible Predicates). *For an action schema $\mathbf{a}$, the set of compatible predicates $\mathcal{P}^{\mathbf{a}}$ contains all predicates whose set of parameters is a subset of those of $\mathbf{a}$:*

$$\mathcal{P}^{\mathbf{a}} := \{\mathbf{p} \in \mathcal{P} \mid \text{s-par}(\mathbf{p}) \subseteq \text{s-par}(\mathbf{a})\},$$

where *s-par* converts the parameters into a set.

Actions. Given an action schema $\mathbf{a}$ and a substitution function ϱ , the corresponding *action* is obtained by replacing each parameter of $\mathbf{a}$ according to ϱ , and is denoted $a = \mathbf{a}[\varrho]$. Actions describe transitions in the state space. An action a is *applicable* in a state s iff $\text{prec}^+(a) \subseteq s$ and $\text{prec}^-(a) \cap s = \emptyset$. Applying an applicable action a in s produces the successor state

$$s' = (s \setminus \text{eff}^-(a)) \cup \text{eff}^+(a),$$

which we denote by $s \rightarrow_a s'$.

Throughout this paper, we use boldface (e.g., $\mathbf{p}$, $\mathbf{a}$) for predicates and action schemas, and regular typeface (e.g., f , a) for facts and actions.

Variation 1: NOTRACE (Domain Only).

1. Infer the intended semantics of each action in the flawed PDDL domain.
2. Propose plausible single-edit repairs: add or remove a positive or negative precondition or effect.
3. Return a concise action overview, the recommended repairs grouped by action, and a justification for each repair.

Constraints.

- Use semantic cues from action and predicate names.
- Introduce no new predicates, variables, or constants; added atoms use arguments available to the action.

Input: a flawed PDDL domain.

Variation 2: WITHTRACE (Domain and Test Traces).

- Follow all NOTRACE steps and constraints.
- Also use test traces as behavioral evidence.
- Positive (whitelist) plans must execute and reach the goal.
- For a negative (blacklist) plan with failure index n , its prefix must be executable, and action n must be inapplicable.

Input: a flawed PDDL domain and positive/negative test traces.

Figure 1: The NOTRACE and WITHTRACE LLM-only prompt abstractions used in our evaluation.

Solutions. Let $\gamma = \langle a_1, \dots, a_k \rangle$ be an action sequence. We write $s \rightarrow_\gamma^* s'$ to denote that s' results from applying γ to s via a state trajectory $\langle s_0, \dots, s_k \rangle$ where $s_0 = s$, $s_k = s'$, and each action is applicable in its preceding state. A solution to a planning problem is an action sequence $\gamma = \langle a_1, \dots, a_k \rangle$ such that $s_I \rightarrow_\gamma^* s'$ for some s' with $s^g \subseteq s'$, and each a_i is a grounding of some action schema $\mathbf{a} \in \mathcal{A}$.

The Repair Problem

We begin by introducing the notation and syntax used to define possible repair operations for a given planning domain. Next, we describe how a set of such repairs can be applied to produce a modified domain. Based on these concepts, we then formalize the domain repair problem in terms of the defined repair operations and a set of positive and negative plans. Our formulation follows the setting introduced by Lin et al. (2025) as closely as possible, while explicitly highlighting the modifications needed to incorporate LLMs for prioritizing semantically meaningful repair sets.

In a planning domain $\mathcal{D} = (\mathcal{P}, \mathcal{A}, \alpha)$, an *atomic repair* is a modification denoted by $r[\mathbf{a}, \mathbf{p}, c, op]$. Here, $\mathbf{a} \in \mathcal{A}$ is an action schema, $\mathbf{p} \in \mathcal{P}$ is a predicate compatible with $\mathbf{a}$, $c \in \{\text{prec}^+, \text{prec}^-, \text{eff}^+, \text{eff}^-\}$ indicates whether the change concerns a positive or negative precondition or effect, and $op \in \{+, -\}$ specifies whether the component is added or removed. We write $\mathcal{D} \Rightarrow_r \mathcal{D}'$ to indicate that applying r to $\mathcal{D}$ yields $\mathcal{D}' = (\mathcal{P}, \mathcal{A}, \alpha')$, where α' results from applying r to α .

A *repair set* δ for a domain is a finite collection of zero or more atomic repairs. We say that δ is *valid* if and only if it contains no two repairs $r, r' \in \delta$ such that one reverses the effect of the other. Specifically, two repairs $r = r[\mathbf{a}, \mathbf{p}, c, op]$ and $r' = r'[\mathbf{a}', \mathbf{p}', c', op']$ are considered to undo each other if $\mathbf{a} = \mathbf{a}'$, $\mathbf{p} = \mathbf{p}'$, $c = c'$, and $op \neq op'$.

Let $\mathcal{D}$ be a domain and δ a valid repair set for $\mathcal{D}$. Applying the repairs in δ in any order yields the same modified domain

$\mathcal{D}'$. We use $\mathcal{D} \Rightarrow_{\delta}^* \mathcal{D}'$ to indicate that $\mathcal{D}'$ is obtained from $\mathcal{D}$ by applying the valid repair set δ .

Definition 2 (Domain Repair Problem). *The domain repair problem is defined as a pair $\mathcal{R} = (\mathcal{D}, \mathbb{T})$, where $\mathcal{D}$ denotes a planning domain and $\mathbb{T} = \{\mathbf{T}_1, \dots, \mathbf{T}_n\}$ for some $n \in \mathbb{N}$. Each element $\mathbf{T}_i$ is a triple $(\Pi_i, \mathbb{P}_i, \mathbb{E}_i)$. Here, $\Pi_i = (\mathcal{D}, \mathcal{T}_i)$ denotes the planning problem; $\mathbb{P}_i$ is a finite, nonempty set of positive plans π_k^+ for Π_i ; and $\mathbb{E}_i$ is a finite, nonempty set of pairs (π_k^-, i_k) associated with Π_i . Each π_k^- denotes a negative plan (a sequence of actions considered undesirable) for Π_i , and i_k is at most the length of that plan.*

Definition 3 (Solution to the Repair Problem). *A solution to $\mathcal{R}$ is a valid repair set δ that transforms the original domain $\mathcal{D}$ into a modified domain $\mathcal{D}'$ through the sequence of repair operations $\mathcal{D} \Rightarrow_{\delta}^* \mathcal{D}'$. This repair must satisfy the following conditions: for every index i with $1 \leq i \leq n$, all positive plans $\pi^+ \in \mathbb{P}_i$ must be valid solutions to the updated planning problem $\Pi'_i = (\mathcal{D}', \mathcal{T}_i)$, meaning that they are executable and achieve the goal; and for each pair $(\pi_k^-, i_k) \in \mathbb{E}_i$, π_k^- must not be a valid plan for Π'_i , with the action at position i_k being the first that cannot be applied.*

Solving the Repair Problem

In this section, we briefly review our symbolic baseline and then introduce our LLM-only approach.

The Baseline Approach

The symbolic baseline (Lin, Grastien, and Bercher 2023; Lin et al. 2025) uses a sound algorithm based on conditional hitting sets to solve the domain repair problem. The authors report runtime and evaluate precision and recall against known ground-truth repairs. To obtain the ground truth, they perturb IPC domains by randomly adding or removing preconditions and effects, which allows precision and recall to be computed directly.

The algorithm executes all whitelist and blacklist test plans. For each failing test, it identifies the possible repairs that could resolve the failure and encodes them as a conditional hitting-set problem. The solver then selects a minimal repair set, applies it to the domain, and re-evaluates the tests under the modified model. If additional tests fail, the newly identified repair requirements are incorporated and the process continues iteratively until a minimal repair set satisfying all tests is found.

The baseline does not exploit semantic cues encoded by the modeler in predicate, action, or domain names within the PDDL file. Its hitting-set solver optimizes only the size of the repair set; when multiple diagnoses share the same cardinality, it returns an arbitrary one. Moreover, the ground-truth repair need not be cardinality-minimal, so it can be missed under this objective.

The LLM-Only Approach

In our previous work (Karimi Bavandpour and Bercher 2025), we introduced several ideas on how LLMs can be exploited to improve purely symbolic repair approaches, including the LLM-only approach studied here. In this ap-

proach, an LLM directly predicts a repair set without symbolic post-processing. In that study, we evaluated only a single LLM, limiting the scope of the study. Here, we extend the results for that approach using more recent LLMs. The present study evaluates whether LLM reasoning can usefully complement symbolic repair. It also examines whether current LLMs are strong enough for the LLM-only approach to produce repairs of sufficient overall quality.

Specifically, we evaluate the two prompts summarized in Figure 1. The NOTRACE prompt supplies only the corrupted domain and asks the LLM to infer intended preconditions and effects from the action and predicate names. The WITHTRACE prompt additionally supplies the positive and negative test traces and instructs the LLM to choose repairs that make every positive trace executable and goal-achieving and every negative trace fail at its designated action.

The LLM-only WITHTRACE setting directly addresses the repair problem because the LLM has access to the test constraints, although its output may still be noisy or unsound. In contrast, NOTRACE does not have access to the tests and therefore addresses a relaxed variant of the problem in which no test constraints must be satisfied. Consequently, multiple semantically plausible repairs may be proposed for any action without regard to whether they solve the actual repair instance.

Experiments

We evaluate our LLM-only approach on the benchmark suite introduced by Lin et al. (2025) and later used in our previous study (Karimi Bavandpour and Bercher 2025), enabling comparison under the same precision, recall, and F_1 metrics. For a ground-truth repair set G and predicted repair set P , precision is $|P \cap G|/|P|$, recall is $|P \cap G|/|G|$, and F_1 is their harmonic mean, with all three scores defined as 0 when P is empty. We additionally report the test pass rate (TR), the fraction of tests satisfied by the predicted repair. TR is particularly important because a solution to the domain repair problem must satisfy every test: high F_1 without near-perfect TR does not provide the required correctness and is therefore insufficient for a reliable stand-alone solver. Each domain represents one repair problem, for which we perform one experimental run; all LLM calls were made through OpenRouter¹. If a call fails or its response cannot be parsed as a repair set, we repeat the same request up to three additional times, and an instance that still has no valid response is treated as an empty prediction. These retries recover failed calls rather than constituting additional experimental runs. The AVG and STD rows report, respectively, the unweighted mean and standard deviation over the per-domain results, so the reported averages are macro averages.

We omit the older versions of the Logistics and Woodworking domains (LOGISTICS98 and WOODWORKING08), retaining LOGISTICS00 and WOODWORKING11. We also omit MPRIME because that domain is designed to use misleading names, making it unsuitable for evaluating an LLM’s ability to exploit semantic cues. Because the symbolic method is stochastic, we repeat in Ta-

¹<https://openrouter.ai/>

LLM Model Name	Default Effort	High Effort	Release Date	Active Par. (B)	Total Par. (B)	Default Effort				High Effort			
						Pr	Re	F_1	TR	Pr	Re	F_1	TR
Symbolic Baseline	-	-	-	-	-	.69	.41	.49	1*	-	-	-	-
OpenAI GPT-4o	Unspec.	-	24-05	NA	NA	.48	.38	.38	.27	-	-	-	-
Phi-4	Unspec.	-	24-12	14	14	.03	.05	.04	.10	-	-	-	-
Qwen3-30B-A3B	Unspec.	-	25-04	3.3	30.5	.46	.32	.34	.23	-	-	-	-
Qwen3-32B	Unspec.	Enabled	25-04	32.8	32.8	.52	.46	.47	.50	.41	.51	.40	.58
OpenAI GPT-OSS-120B	Medium	High	25-08	5.1	117	.56	.58	.53	.50	.34	.44	.37	.40
OpenAI GPT-OSS-20B	Medium	High	25-08	3.6	21	.42	.51	.42	.27	.47	.60	.50	.50
Minstral 3 14B	Unspec.	-	25-12	14	14	.23	.34	.27	.12	-	-	-	-
Mistral Large 3	Unspec.	-	25-12	41	675	.29	.37	.31	.30	-	-	-	-
Mistral Small 4	Off	High	26-03	6.5	119	.14	.15	.13	.15	.45	.51	.41	.36
NVIDIA Nemotron 3 Super	Medium	-	26-03	12	120	.40	.59	.43	.33	-	-	-	-
DeepSeek V4 Flash	Unspec.	X-High	26-04	13	284	.57	.73	.60	.56	.67	.83	.70	.72
DeepSeek V4 Pro	Unspec.	X-High	26-04	49	1600	.91	.76	.80	.69	.61	.89	.69	.65
Mistral Medium 3.5	Unspec.	High	26-04	128	128	.34	.45	.35	.42	.44	.72	.52	.52
GLM 5.2	High	X-High	26-06	40	753	.90	.82	.85	.85	.93	.83	.87	.82
NVIDIA Nemotron 3 Ultra	High	-	26-06	55	550	.63	.76	.65	.74	-	-	-	-

LLM Model Name	Default Effort	High Effort	Release Date	Active Par. (B)	Total Par. (B)	Default Effort				High Effort			
						Pr	Re	F_1	TR	Pr	Re	F_1	TR
OpenAI GPT-4o	Unspec.	-	24-05	NA	NA	.43	.29	.33	.31	-	-	-	-
Phi-4	Unspec.	-	24-12	14	14	0	0	0	.07	-	-	-	-
Qwen3-30B-A3B	Unspec.	-	25-04	3.3	30.5	.25	.15	.18	.10	-	-	-	-
Qwen3-32B	Unspec.	Enabled	25-04	32.8	32.8	.26	.22	.22	.23	.12	.10	.11	.11
OpenAI GPT-OSS-120B	Medium	High	25-08	5.1	117	.67	.45	.49	.57	.15	.16	.15	.09
OpenAI GPT-OSS-20B	Medium	High	25-08	3.6	21	.26	.24	.23	.26	.38	.47	.41	.55
Minstral 3 14B	Unspec.	-	25-12	14	14	.09	.14	.10	.17	-	-	-	-
Mistral Large 3	Unspec.	-	25-12	41	675	.44	.42	.35	.30	-	-	-	-
Mistral Small 4	Off	High	26-03	6.5	119	.10	.11	.09	.11	.55	.49	.44	.50
NVIDIA Nemotron 3 Super	Medium	-	26-03	12	120	.57	.27	.34	.48	-	-	-	-
DeepSeek V4 Flash	Unspec.	X-High	26-04	13	284	.69	.64	.63	.75	.81	.69	.71	.81
DeepSeek V4 Pro	Unspec.	X-High	26-04	49	1600	.85	.71	.74	.82	.85	.83	.82	.92
Mistral Medium 3.5	Unspec.	High	26-04	128	128	.42	.44	.36	.46	.74	.58	.62	.75
GLM 5.2	High	X-High	26-06	40	753	.94	.70	.78	.82	.94	.76	.82	.85
NVIDIA Nemotron 3 Ultra	High	-	26-06	55	550	.95	.66	.74	.78	-	-	-	-

Table 1: The top table shows the NOTRACE and the bottom one shows the WITHTRACE LLM-only results. Each row reports the metrics averaged over the 12 error-injected IPC domains described in the text. Parameter counts are given in billions. Pr, Re, and F_1 denote precision, recall, and their harmonic mean, respectively; TR is the average test pass rate, i.e., the fraction of tests satisfied by the repairs. The grayed-out first row of the top table gives the symbolic-only baseline, whose TR is 1 by construction and is marked 1*. *Default Effort* uses the model’s default reasoning configuration, while *High Effort* uses the enhanced configuration specified in that column. *Unspec.* means default reasoning enablement is not advertised. A “-” in an effort column means no alternative configuration is supported; elsewhere, it denotes an inapplicable result.

Prompt	Default Effort			High Effort			
	F_1	TR	Model	F_1	TR	Model	
NoTRACE	AVG	.85	.85	GLM-5.2	.87	.82	GLM-5.2
	STD	.17	.28		.12	.29	
WithTRACE	AVG	.78	.82	GLM-5.2	.82	.92	DSV4-P
	STD	.22	.29		.17	.26	

Table 2: Average F_1 and TR for each trace-inclusion and reasoning-effort setting, with the corresponding best-performing model. These metrics are reported as in Table 1; see its caption for their definitions. DSV4-P: DeepSeek V4 Pro; GLM-5.2: GLM 5.2. For High Effort with WITHTRACE, GLM-5.2 matches DSV4-P’s mean F_1 , but DSV4-P is shown because its F_1 standard deviation is lower (.17 versus .18). Per-domain breakdowns are reported in Tables 3, 4, 5, and 6.

ble 1 the five-run average published by Lin et al. (2025) instead of rerunning it.

Table 1 presents our main results for the recent models. We repeat the symbolic result in its first row only to provide a common point of reference. Compared with the NOTRACE prompt used for the earlier runs reported in our previous work (Karimi Bavandpour and Bercher 2025), the NOTRACE prompt in these experiments is shorter: it neither requests an explicit reasoning trace nor provides a one-shot example. We made this change to keep the task and its context simpler for smaller models, whose performance is a central focus of this evaluation. The WITHTRACE prompt minimally extends NOTRACE by supplying the whitelist and blacklist test traces and asking the LLM to satisfy all tests.

Our previous work reports a mean F_1 of .46 for GPT-4o with the original NOTRACE prompt (Karimi Bavandpour and Bercher 2025), compared with .38 using the shorter prompt in Table 1. With test traces excluded and default reasoning, GLM 5.2 achieves the highest mean F_1 of .85. The release dates in Table 1 also show a pronounced generational improvement: the best 2025 model reaches $F_1 = .53$, whereas the best 2026 model reaches $F_1 = .85$.

Table 2 summarizes the interaction between trace inclusion and reasoning effort. Supplying the test traces does not improve the best result at default effort: GLM 5.2 remains the best model, but its mean F_1 falls from .85 to .78. Reasoning jointly over many potentially long plans is combinatorial and can also consume or exceed the LLM’s context window, which may explain why the additional evidence is not consistently useful. The per-domain results support this interpretation: Table 3 contains a numeric result for every model-domain pair, whereas Table 4 contains both context-limit errors and invalid or unparseable outputs.

Higher reasoning effort helps several models, but not all of them. Without test traces, Table 1 shows that GLM 5.2 improves from $F_1 = .85$ to .87. The increase is larger for Mistral Small 4, from .13 to .41. Nevertheless, Table 2 shows that providing test traces at high effort yields a best mean F_1 of only .82, shared by DeepSeek V4 Pro and GLM 5.2. This

is below the best default-effort NOTRACE result of .85.

Test satisfaction exposes a more consequential limitation. The strongest LLM result, GLM 5.2 with high effort and NOTRACE, improves substantially on the symbolic baseline in F_1 (.87 versus .49), but its mean TR is only .82. Its TR falls to .06 on Thoughtful, meaning that only 6% of that domain’s tests are satisfied. Thus, high repair-set overlap does not provide the strict correctness guarantee required by the repair problem.

Reasoning over the test traces does help with their combinatorial constraints in one important case: for DeepSeek V4 Pro with WITHTRACE, increasing the reasoning effort raises mean TR from .82 to .92 while also raising F_1 from .74 to .82. Nevertheless, .92 still falls short of complete test satisfaction, and TR again drops to .06 on Thoughtful. Recent open-weight models therefore improve semantic repair quality without guaranteeing that the predicted repair is a solution. These findings are supported only by our error-injected IPC benchmark, which is publicly available online². The models may therefore have encountered the underlying domains or some of their fixes during training and could be recalling them rather than deriving every repair from the supplied instance. This possible benchmark contamination limits how strongly the observed F_1 and TR gains can be expected to generalize.

We argue for hybrid methods that use an LLM to rank or filter semantically promising candidates while a symbolic component provides theoretical test-satisfaction guarantees, as proposed in our previous work (Karimi Bavandpour and Bercher 2025); realizing such a method remains future work.

Conclusion & Future Work

Our results convey two main messages. First, recent open-weight LLMs can use semantic cues to identify repairs that agree much more closely with the ground truth than the symbolic baseline: the best mean F_1 is .87, compared with .49. Reasoning effort can strengthen this ability, but its effect is model-dependent, and supplying test traces does not improve the best F_1 . Second, repair-set overlap is not a correctness guarantee. The setting with the best F_1 has a mean TR of .82 and a TR of only .06 on Thoughtful; even the strongest trace-aware setting reaches a mean TR of .92 and again only .06 on Thoughtful. Current LLM-only methods therefore improve semantic repair quality but cannot reliably solve the repair problem, as every test must pass.

A promising direction is consequently to combine the complementary strengths of both approaches: an LLM can express semantic repair preferences, while a symbolic reasoner preserves theoretical test-satisfaction guarantees (Karimi Bavandpour and Bercher 2025). We will also isolate the effects of prompt length, explicit reasoning requests, and examples. Finally, because the evaluated IPC-derived benchmark is public, possible training-data recall limits the generality of our results; evaluation on novel, unpublished domains is needed to establish whether these gains transfer to genuinely unseen repair problems.

²<https://zenodo.org/records/14533200> (Lin et al. 2024)

Acknowledgments

Pascal Bercher is the recipient of an Australian Research Council (ARC) Discovery Early Career Researcher Award (DECRA), project number DE240101245, funded by the Australian Government.

References

- Aineto, D.; Jiménez, S.; and Onaindia, E. 2018. Learning STRIPS Action Models with Classical Planning. In *Proc. of the 28th ICAPS*, 399–407. AAAI Press.
- Bercher, P.; Sreedharan, S.; and Vallati, M. 2025. A Survey on Model Repair in AI Planning. In *Proc. of the 34th IJCAI*, 10371–10380. IJCAI.
- Beriachvili, N.; and Bit-Monnot, A. 2026. A Constraint Formulation for Domain Repair with Ground or Lifted Test Plans. In *Proc. of the 36th ICAPS*, 20–28. AAAI Press.
- Gragera, A.; Fuentetaja, R.; Olaya, Á. G.; and Fernández, F. 2025. On the Gains from Using Action Observations in Domain Repair. In *Proc. of the 35th ICAPS*, 343–347. AAAI Press.
- Guan, L.; Valmeekam, K.; Sreedharan, S.; and Kambhampati, S. 2023. Leveraging Pre-trained Large Language Models to Construct and Utilize World Models for Model-based Task Planning. In *Proc. of the 37th NeurIPS*. Neural Information Processing Systems Foundation, Inc.
- Huang, S.; Cohn, T.; and Lipovetzky, N. 2025. Chasing progress, not perfection: Revisiting strategies for end-to-end LLM plan generation. In *Proc. of the 35th ICAPS*, 204–212. AAAI Press.
- Huang, S.; Lipovetzky, N.; and Cohn, T. 2025. Planning in the Dark: LLM-Symbolic Planning Pipeline Without Experts. In *Proc. of the 39th AAAI*, 26542–26550. AAAI Press.
- Karimi Bavandpour, N.; and Bercher, P. 2025. Finding Semantically Guided Repairs in PDDL Domains Using LLMs. In *ICAPS 2025 Workshop on Human-Aware and Explainable Planning (HAXP)*.
- Katz, M.; Kokel, H.; Muise, C.; Sohrabi, S.; and Sreedharan, S. 2025. Make Planning Research Rigorous Again! *CoRR*, abs/2505.21674.
- Lin, S.; and Bercher, P. 2021. Change the World – How Hard Can that Be? On the Computational Complexity of Fixing Planning Models. In *Proc. of the 30th IJCAI*, 4152–4159. IJCAI.
- Lin, S.; Grastien, A.; and Bercher, P. 2023. Towards Automated Modeling Assistance: An Efficient Approach for Repairing Flawed Planning Domains. In *Proc. of the 37th AAAI*, 12022–12031. AAAI Press.
- Lin, S.; Grastien, A.; Shome, R.; and Bercher, P. 2024. Experimental Results for the AAAI 2025 Paper: “Told You That Will Not Work: Optimal Corrections to Planning Domains Using Counter-Example Plans”. <https://zenodo.org/records/14533200>.
- Lin, S.; Grastien, A.; Shome, R.; and Bercher, P. 2025. Told You That Will Not Work: Optimal Corrections to Planning Domains Using Counter-Example Plans. In *Proc. of the 39th AAAI*, 26596–26604. AAAI Press.
- Miller, T. 2019. Explanation in artificial intelligence: Insights from the social sciences. *AIJ*, 267: 1–38.
- Oswald, J. T.; Srinivas, K.; Kokel, H.; Lee, J.; Katz, M.; and Sohrabi, S. 2024. Large Language Models as Planning Domain Generators. In *Proc. of the 34th ICAPS*, 423–431. AAAI Press.
- Tantakoun, M.; Muise, C.; and Zhu, X. 2025. LLMs as Planning Formalizers: A Survey for Leveraging Large Language Models to Construct Automated Planning Models. In *Findings of the Association for Computational Linguistics: ACL 2025*, 25167–25188. Association for Computational Linguistics.

Appendix: Per-Domain Results

Tables 3, 4, 5, and 6 provide the per-domain results for open-weight models:

DSV4-F (DeepSeek V4 Flash), DSV4-P (DeepSeek V4 Pro), GLM-5.2 (GLM 5.2), OSS20B (gpt-oss-20B), OSS120B (gpt-oss-120B), Minst-14B (Minstral 3 14B), MistS-4 (Mistral Small 4), MistM-3.5 (Mistral Medium 3.5), MistL-3 (Mistral Large 3), Nem3-S (NVIDIA Nemotron 3 Super), Nem3-U (NVIDIA Nemotron 3 Ultra), Phi4 (Phi-4), Qwen3-30B (Qwen3-30B-A3B), Qwen3-32B. Best values per row are bold; the final two rows report the mean and standard deviation across domains.

	OSS20B				OSS120B				Minst-14B				MistL-3				Phi4				Qwen3-30B				Qwen3-32B			
Domain	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR
Floortile	.67	.50	.57	.17	.67	1	.80	.83	.60	.75	.67	.17	.40	.50	.44	0	0	0	0	.50	.50	.50	0	1	.75	.86	.17	
Freecell	0	0	0	0	.25	.50	.33	.21	.14	.25	.18	.01	.14	.25	.18	0	0	0	0	.40	.50	.44	.21	.67	.50	.57	.21	
GED	.10	.10	.10	0	1	.20	.33	.55	.12	.20	.15	0	.18	.20	.19	0	0	0	0	.67	.20	.31	.31	.20	.10	.13	0	
Hiking	.80	1	.89	.22	.75	.75	.75	.22	.33	.50	.40	0	.75	.75	.75	1	0	0	0	.78	.67	.50	.57	.22	.75	.75	.75	1
Logistics00	.67	1	.80	1	.67	1	.80	1	.17	.25	.20	0	.50	.50	.50	0	0	0	0	.03	.67	.50	.57	0	1	.75	.86	.97
Scanalyzer	.17	1	.29	0	.67	1	.80	1	.25	.50	.33	.40	0	0	0	0	0	0	0	0	0	0	0	0	.20	.50	.29	.93
Slitherlink	1	.50	.67	.33	0	0	0	0	.33	.50	.40	.33	.33	.50	.40	.67	.25	.50	.33	.33	0	0	0	0	.50	.50	.50	.33
Sokoban	.25	.50	.33	.40	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	.33	.50	.40	.40	0	0	0	0
Tetris	.67	.50	.57	.50	.09	.25	.13	.60	.20	.50	.29	0	.19	.75	.30	.50	0	0	0	0	.29	.50	.36	.50	.22	.50	.31	.50
Thoughtful	.09	.10	.10	0	.44	.40	.42	.06	.29	.20	.24	0	.50	.40	.44	0	.12	.10	.11	.06	.33	.10	.15	0	.17	.20	.18	0
Tidybot	.12	.08	.10	.22	.38	.25	.30	.11	.14	.08	.11	.44	.17	.08	.11	.67	0	0	0	0	.67	.17	.27	.44	.50	.33	.40	1
Woodwork11	.45	.83	.59	.38	.80	.67	.73	.38	.22	.33	.27	.12	.30	.50	.37	.75	0	0	0	0	1	.33	.50	.62	1	.67	.80	.88
AVG	.42	.51	.42	.27	.56	.58	.53	.50	.23	.34	.27	.12	.29	.37	.31	.30	.03	.05	.04	.10	.46	.32	.34	.23	.52	.46	.47	.50
STD	.32	.37	.29	.28	.32	.35	.30	.37	.14	.21	.16	.17	.21	.25	.21	.37	.07	.14	.09	.22	.28	.20	.19	.22	.35	.25	.29	.41

	DSV4-F				DSV4-P				GLM-5.2				MistS-4				MistM-3.5				Nem3-S				Nem3-U			
Domain	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR	Pr	Re	F ₁	TR
Floortile	1	1	1	1	1	1	1	1	1	1	1	1	.50	.50	.50	0	.67	1	.80	.83	.80	1	.89	1	.67	1	.80	1
Freecell	.25	.50	.33	.07	1	.75	.86	.51	1	1	1	1	0	0	0	0	.12	.25	.17	0	.04	.50	.07	0	.33	.50	.40	.01
GED	.75	.30	.43	0	.80	.40	.53	.59	.83	.50	.62	.69	0	0	0	.55	.04	.10	.05	0	.22	.20	.21	.31	.75	.30	.43	.69
Hiking	.43	.75	.55	0	1	.75	.86	1	1	.75	.86	1	.33	.50	.40	0	1	.75	.86	1	.29	.50	.36	0	1	1	1	1
Logistics00	.67	1	.80	1	1	1	1	1	.80	1	.89	1	.25	.25	.25	.03	0	0	0	0	.75	.75	.75	.03	.67	1	.80	1
Scanalyzer	.33	1	.50	1	1	1	1	1	1	1	1	1	0	0	0	0	.17	.50	.25	.93	.29	1	.44	1	.33	1	.50	1
Slitherlink	.50	1	.67	1	1	1	1	1	1	1	1	1	0	0	0	0	.17	.50	.25	.33	1	1	1	1	.50	1	.67	1
Sokoban	1	1	1	1	1	.50	.67	.60	1	1	1	1	0	0	0	0	0	0	0	0	.50	.50	.50	.60	1	1	1	1
Tetris	1	.50	.67	.50	1	.50	.67	.50	1	.50	.67	.50	0	0	0	.10	.27	.75	.40	0	.11	.50	.18	0	.75	.75	.75	1
Thoughtful	.12	.10	.11	0	.83	.50	.62	0	.62	.50	.56	.06	0	0	0	0	.26	.60	.36	.06	.06	.10	.07	0	.67	.60	.63	.06
Tidybot	.44	.67	.53	1	.73	.67	.70	1	.70	.58	.64	1	.50	.17	.25	.44	.38	.25	.30	1	.40	.17	.24	0	.44	.33	.38	1
Woodwork11	.40	1	.57	.12	.50	1	.67	.12	.86	1	.92	1	.15	.33	.21	.62	1	.67	.80	.88	.36	.83	.50	0	.40	.67	.50	.12
AVG	.57	.73	.60	.56	.91	.76	.80	.69	.90	.82	.85	.85	.14	.15	.13	.15	.34	.45	.35	.42	.40	.59	.43	.33	.63	.76	.65	.74
STD	.29	.31	.25	.46	.15	.23	.17	.35	.13	.22	.17	.28	.19	.19	.17	.23	.34	.31	.30	.44	.29	.32	.30	.42	.22	.26	.21	.40

Table 3: Per-domain results using the NOTRACE prompt in Figure 1; test traces are not sent to the LLM. The default effort setting is used for each LLM. TR is the test pass rate, i.e., the fraction of tests satisfied by the repair; the AVG row averages this rate across domains. For each domain and the AVG row, the best precision, recall, F_1 , and TR values across all models are shown in bold. The upper and lower parts of the table contain models released in 2025 and 2026, respectively.

Domain	OSS20B				OSS120B				Minst-14B				MistL-3				Phi4				Qwen3-30B				Qwen3-32B			
	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR
Floortile	.67	.50	.57	1	1	.75	.86	1	.25	.25	.25	.83	1	.75	.86	1	0	0	0	.83	.67	.50	.57	0	.50	.50	.50	.83
Freecell	e_1	e_1	e_1	0	e_1	e_1	e_1	0	0	0	0	.01	.03	.75	.05	.01	e_1	e_1	e_1	0	e_1	e_1	e_1	0	e_1	e_1	e_1	0
GED	.07	.10	.08	.31	.50	.10	.17	.31	0	0	0	0	1	.10	.18	0	0	0	0	0	0	0	0	0	0	0	0	0
Hiking	.67	.50	.57	1	1	.75	.86	1	.17	.25	.20	0	.50	.50	.50	.22	0	0	0	0	1	.50	.67	.22	.67	.50	.57	.22
Logistics00	.50	.50	.50	.03	1	1	1	1	.14	.25	.18	.03	.75	.75	.75	.03	e_1	e_1	e_1	0	.33	.25	.29	0	0	0	0	0
Scanalyzer	.14	.50	.22	.07	1	.50	.67	.93	.09	.50	.15	.93	.20	.50	.29	.93	e_1	e_1	e_1	0	1	.50	.67	.93	0	0	0	0
Slitherlink	1	.50	.67	.67	1	1	1	1	0	0	0	0	.50	.50	.50	.33	0	0	0	0	0	0	0	0	.33	.50	.40	.33
Sokoban	0	0	0	0	.33	.50	.40	.40	0	0	0	0	0	0	0	0	e_1	e_1	e_1	0	0	0	0	0	1	.50	.67	.40
Tetris	.08	.25	.12	0	.50	.25	.33	.50	0	0	0	0	.11	.25	.15	0	e_1	e_1	e_1	0	0	0	0	0	.25	.50	.33	.50
Thoughtful	0	0	0	0	.02	.10	.04	.06	.08	.10	.09	.06	.05	.20	.08	0	e_1	e_1	e_1	0	e_3	e_3	e_3	0	0	0	0	0
Tidybot	0	0	0	0	1	.08	.15	.56	0	0	0	0	.83	.42	.56	1	e_1	e_1	e_1	0	0	0	0	0	.33	.17	.22	.44
Woodwork11	0	0	0	0	.67	.33	.44	.12	.33	.33	.33	.12	.25	.33	.29	.12	e_1	e_1	e_1	0	0	0	0	0	0	0	0	0
AVG	.26	.24	.23	.26	.67	.45	.49	.57	.09	.14	.10	.17	.44	.42	.35	.30	0	0	0	.07	.25	.15	.18	.10	.26	.22	.22	.23
STD	.34	.23	.26	.38	.38	.34	.36	.38	.11	.16	.11	.32	.37	.24	.27	.40	0	0	0	.23	.39	.22	.27	.26	.31	.24	.25	.26

Domain	DSV4-F				DSV4-P				GLM-5.2				MistS-4				MistM-3.5				Nem3-S				Nem3-U			
	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR
Floortile	1	1	1	1	1	1	1	1	1	1	1	1	.67	.50	.57	.83	.50	.75	.60	.83	1	.25	.40	.83	1	1	1	1
Freecell	.33	.25	.29	.21	1	.75	.86	.70	1	.75	.86	.70	.02	.25	.04	.51	.33	.50	.40	.21	0	0	0	0	1	.50	.67	.21
GED	.78	.70	.74	1	.75	.30	.43	1	.67	.20	.31	.97	.12	.10	.11	0	1	.10	.18	.31	.50	.10	.17	.31	.75	.30	.43	1
Hiking	1	.75	.86	1	1	.75	.86	1	1	.75	.86	1	0	0	0	0	1	.75	.86	1	.67	.50	.57	.78	1	.75	.86	1
Logistics00	.67	1	.80	1	.67	1	.80	1	1	1	1	1	0	0	0	0	1	.50	.67	0	.60	.75	.67	1	1	1	1	1
Scanalyzer	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	.20	.50	.29	.93	1	.50	.67	.93	1	1	1	1
Slitherlink	.50	1	.67	1	1	1	1	1	1	1	1	1	0	0	0	0	.20	.50	.29	.67	1	.50	.67	.67	1	1	1	1
Sokoban	1	1	1	1	1	.50	.67	.60	1	.50	.67	.60	.33	.50	.40	0	.20	.50	.29	.40	0	0	0	0	1	1	1	1
Tetris	0	0	0	0	1	.50	.67	.50	1	.50	.67	.50	0	0	0	0	.06	.25	.09	.50	e_3	e_3	e_3	0	1	.25	.40	.50
Thoughtful	1	.20	.33	0	.67	.60	.63	.06	.83	.50	.62	.06	0	0	0	0	0	0	0	0	.11	.10	.11	0	1	.50	.67	0
Tidybot	.50	.25	.33	1	.86	.50	.63	1	.80	.33	.47	1	0	0	0	0	.11	.08	.10	.44	1	.08	.15	.56	.60	.25	.35	1
Woodwork11	.50	.50	.50	.75	.27	.67	.38	1	1	.83	.91	1	0	0	0	0	.50	.83	.62	.25	1	.50	.67	.62	1	.33	.50	.62
AVG	.69	.64	.63	.75	.85	.71	.74	.82	.94	.70	.78	.82	.10	.11	.09	.11	.42	.44	.36	.46	.57	.27	.34	.48	.95	.66	.74	.78
STD	.32	.36	.32	.40	.22	.23	.20	.29	.11	.27	.22	.29	.20	.19	.18	.26	.36	.26	.26	.32	.42	.25	.28	.38	.12	.32	.26	.34

Table 4: Per-domain results using the WITHTRACE prompt in Figure 1; test traces are sent to the LLM. The default effort setting is used for each LLM. TR is the test pass rate, i.e., the fraction of tests satisfied by the repair; the AVG row averages this rate across domains. For each domain and the AVG row, the best precision, recall, F_1 , and TR values across all models are shown in bold. The upper part of the table contains models released in 2025, and the lower part contains models released in 2026. Missing values are marked as e_1 for context-limit errors, e_2 for other API-call errors, and e_3 for invalid or unparsable output. Missing data are treated as 0 in our calculations.

Domain	OSS20B				OSS120B				Qwen3-32B			
	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR
Floortile	.67	1	.80	.83	.67	1	.80	.83	.60	.75	.67	.83
Freecell	.14	.50	.22	.21	e_3	e_3	e_3	0	.12	.25	.17	0
GED	.06	.10	.07	0	e_3	e_3	e_3	0	.25	.10	.14	0
Hiking	1	.75	.86	1	1	.75	.86	1	.67	.50	.57	1
Logistics00	1	1	1	1	.80	1	.89	1	1	.75	.86	.97
Scanalyzer	.33	1	.50	1	.11	.50	.18	0	.20	1	.33	1
Slitherlink	1	1	1	1	1	1	1	1	.33	1	.50	1
Sokoban	1	1	1	1	.50	1	.67	1	.33	.50	.40	.40
Tetris	.25	.25	.25	0	e_3	e_3	e_3	0	.20	.50	.29	.50
Thoughtful	.02	.10	.03	0	e_3	e_3	e_3	0	0	0	0	0
Tidybot	0	0	0	0	e_3	e_3	e_3	0	.20	.08	.12	.44
Woodwork11	.20	.50	.29	0	e_3	e_3	e_3	0	1	.67	.80	.88
AVG	.47	.60	.50	.50	.34	.44	.37	.40	.41	.51	.40	.58
STD	.41	.39	.39	.47	.40	.46	.41	.48	.32	.33	.27	.40

Domain	DSV4-F				DSV4-P				GLM-5.2				MistS-4				MistM-3.5			
	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR
Floortile	1	1	1	1	.80	1	.89	1	1	1	1	1	.67	.50	.57	0	.67	1	.80	.83
Freecell	1	.50	.67	.21	1	1	1	1	1	1	1	1	.25	.50	.33	.21	.18	.50	.27	0
GED	.78	.70	.74	.69	.88	.70	.78	.59	1	.50	.67	.55	.50	.20	.29	0	.62	.50	.56	.10
Hiking	.75	.75	.75	1	.33	.75	.46	0	1	.75	.86	1	.17	.75	.27	0	.43	.75	.55	0
Logistics00	.67	1	.80	1	.50	1	.67	1	.67	1	.80	1	.60	.75	.67	1	.50	.75	.60	.03
Scanalyzer	.33	1	.50	1	1	1	1	1	1	1	1	1	.33	1	.50	1	.33	1	.50	1
Slitherlink	1	1	1	1	.10	1	.18	0	1	1	1	1	.33	.50	.40	.33	.50	1	.67	1
Sokoban	.50	1	.67	1	1	1	1	1	1	1	1	1	.33	.50	.40	.60	1	1	1	1
Tetris	.75	.75	.75	.50	.50	.75	.60	1	1	.75	.86	.50	.20	.50	.29	0	.11	.50	.17	1
Thoughtful	.28	.70	.40	.06	.33	.70	.45	.06	1	.60	.75	.06	.25	.30	.27	.06	.22	.40	.29	.06
Tidybot	.67	.67	.67	1	.53	.75	.62	1	.75	.75	.75	1	.75	.25	.38	.44	.17	.42	.24	.44
Woodwork11	.29	.83	.43	.12	.40	1	.57	.12	.80	.67	.73	.75	1	.33	.50	.62	.50	.83	.62	.75
AVG	.67	.83	.70	.72	.61	.89	.69	.65	.93	.83	.87	.82	.45	.51	.41	.36	.44	.72	.52	.52
STD	.26	.16	.18	.37	.30	.13	.25	.44	.12	.18	.12	.29	.25	.22	.12	.36	.25	.24	.24	.43

Table 5: Per-domain results using the NOTRACE prompt in Figure 1; test traces are not sent to the LLM. The high-effort setting is used for each LLM when available (see Table 1); models without this option are omitted. TR is the test pass rate, i.e., the fraction of tests satisfied by the repair; the AVG row averages this rate across domains. For each domain and the AVG row, the best precision, recall, F_1 , and TR values across all models are shown in bold. The upper part of the table contains models released in 2025, and the lower part contains models released in 2026. Missing values are marked as e_1 for context-limit errors, e_2 for other API-call errors, and e_3 for invalid or unparsable output. Missing data are treated as 0 in our calculations.

Domain	OSS20B				OSS120B				Qwen3-32B			
	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR
Floortile	1	1	1	1	e_3	e_3	e_3	0	.25	.25	.25	0
Freecell	e_1	e_1	e_1	0	e_1	e_1	e_1	0	e_1	e_1	e_1	0
GED	.06	.10	.08	.55	e_3	e_3	e_3	0	0	0	0	0
Hiking	.60	.75	.67	1	e_3	e_3	e_3	0	.67	.50	.57	1
Logistics00	.75	.75	.75	1	.50	.75	.60	.03	0	0	0	.03
Scanalyzer	.33	1	.50	1	1	1	1	1	0	0	0	0
Slitherlink	1	1	1	1	e_3	e_3	e_3	0	.50	.50	.50	.33
Sokoban	.33	.50	.40	.60	e_3	e_3	e_3	0	0	0	0	0
Tetris	.50	.50	.50	.50	e_3	e_3	e_3	0	0	0	0	0
Thoughtful	0	0	0	0	.33	.20	.25	.06	e_2	e_2	e_2	0
Tidybot	0	0	0	0	e_3	e_3	e_3	0	0	0	0	0
Woodwork11	0	0	0	0	e_3	e_3	e_3	0	0	0	0	0
AVG	.38	.47	.41	.55	.15	.16	.15	.09	.12	.10	.11	.11
STD	.37	.41	.37	.43	.30	.33	.31	.27	.22	.19	.20	.28

Domain	DSV4-F				DSV4-P				GLM-5.2				MistS-4				MistM-3.5			
	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR	Pr	Re	F_1	TR
Floortile	1	1	1	1	.67	1	.80	1	1	1	1	1	1	.75	.86	.83	.67	1	.80	1
Freecell	1	.50	.67	.21	.80	1	.89	1	1	1	1	1	.02	.50	.03	.01	e_3	e_3	e_3	0
GED	.83	.50	.62	1	.80	.40	.53	1	.80	.40	.53	1	0	0	0	0	.75	.30	.43	1
Hiking	1	.75	.86	1	1	.75	.86	1	1	.75	.86	1	.75	.75	.75	1	.75	.75	.75	1
Logistics00	.80	1	.89	1	.67	1	.80	1	.80	1	.89	1	.75	.75	.75	1	1	1	1	1
Scanalyzer	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Slitherlink	1	1	1	1	1	1	1	1	1	.50	.67	.67	.50	1	.67	1	1	.50	.67	.67
Sokoban	1	1	1	1	1	1	1	1	1	1	1	1	1	.50	.67	.60	1	1	1	1
Tetris	1	.25	.40	.50	1	.75	.86	1	1	.50	.67	.50	0	0	0	0	1	.50	.67	.50
Thoughtful	.27	.30	.29	.06	.55	.60	.57	.06	.89	.80	.84	.06	1	.20	.33	0	.09	.10	.10	0
Tidybot	.57	.33	.42	1	.67	.50	.57	1	.80	.33	.47	1	.50	.08	.14	.56	.67	.17	.27	1
Woodwork11	.27	.67	.38	1	1	1	1	1	1	.83	.91	1	.08	.33	.13	0	1	.67	.80	.88
AVG	.81	.69	.71	.81	.85	.83	.82	.92	.94	.76	.82	.85	.55	.49	.44	.50	.74	.58	.62	.75
STD	.27	.29	.27	.33	.17	.22	.17	.26	.09	.25	.18	.29	.41	.35	.36	.44	.34	.36	.33	.37

Table 6: Per-domain results using the WITHTRACE prompt in Figure 1; test traces are sent to the LLM. The high-effort setting is used for each LLM when available (see Table 1); models without this option are omitted. TR is the test pass rate, i.e., the fraction of tests satisfied by the repair; the AVG row averages this rate across domains. For each domain and the AVG row, the best precision, recall, F_1 , and TR values across all models are shown in bold. The upper part of the table contains models released in 2025, and the lower part contains models released in 2026. Missing values are marked as e_1 for context-limit errors, e_2 for other API-call errors, and e_3 for invalid or unparsable output. Missing data are treated as 0 in our calculations.